Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 7
Расчёт параметров модели ПО «Модель ОС»
По дисциплине “Операционные системы”

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель: Провести анализ работы ПО «Модель ОС» в однопрограммном и мультипрограммном режимах.

Исходники: https://github.com/kukirmash/os_model/

Задание

1. Проанализировать в виде таблицы расчёт времени T_mono и T_multi по составляющим.

2. Внести изменения в программу «Модель ОС»:

- добавить подпрограмму «Увеличить время готовых процессов» на заданное число тактов;

- сделать вызов этой подпрограммы в требуемых местах;

- добавить вычисление всех параметров (по одному, с постоянной проверкой правильности вычислений).

3. Путём прогона простых тестов проверить верность программы. Записать результаты прогона и сравнить с предполагаемыми.

В отчёте представить: блок-схемы алгоритмов, анализ программного кода, листинг программы, экранные формы.

Расчёт времени выполнения задания в двух режимах

В однопрограммной ОС задание выполняется без конкуренции за процессор, поэтому время его выполнения складывается из времени загрузки, выполнения команд, ввода-вывода и завершения. В мультипрограммной системе к этим составляющим добавляются затраты на системные действия и время ожидания в очереди готовности. Составляющие времени процесса приведены в таблице 1.

Составляющая времени	**Обозначение**	**T_mono**	**T_multi**	**Поле PSW / источник**
Загрузка задания	T_загр	+	+	LoadTicks (Costs.Load)
Выполнение на ЦПр	T_ЦПр	+	+	ExecTicks
Инициализация и блокировка ВВ	T_ВВ	+	+	IOTicks
Завершение задания	T_заверш	+	+	TermTicks (Costs.Terminate)

Составляющая времени	**Обозначение**	**T_mono**	**T_multi**	**Поле PSW / источник**
Переключение «Готов» → «Активен»	T_перекл	−	+	SwitchTicks (Costs.Dispatch)
Логическая инициализация ВВ	T_иниц.ВВ	−	+	IOInitTicks (Costs.IOInit)
Обслуживание прерывания ВВ	T_прерыв	−	+	IOIntrTicks (IOInterrupt)
Ожидание в очереди готовности	T_ожид	−	+	WaitTicks + SystemWaitTicks
Переключение «Активен» → «Готов»	—	−	−	в варианте 11 нет вытеснения
Блокировка по обращению к памяти	—	−	−	в модели не возникает

Время выполнения задания в однопрограммной ОС:

$$T_{mono} = T_{загр} + T_{ЦПр} + T_{ВВ} + T_{заверш}.$$

Время нахождения задания в мультипрограммной системе:

$$T_{multi} = T_{mono} + T_{перекл} + T_{иниц.ВВ} + T_{прерыв} + T_{ожид}.$$

Показатели, характеризующие отдельный процесс:

$$D_{exe} = T_{multi} / T_{mono} \cdot 100\ \%$$ (увеличение времени выполнения),

$$D_{ready} = T_{ожид} / T_{multi} \cdot 100\ \%$$ (доля ожидания в T_multi).

Эффективное время работы модели складывается из модельных тактов и времени системных действий:

$$T_{эфф} = T_{мод} + T_{сист}.$$

Среднее время выполнения задания в однопрограммной ОС и возможное число выполненных заданий:

$$T_{mono_avg} = \Sigma\, T_{mono} / M_{multi},$$

$$M_{mono} = T_{эфф} / T_{mono_avg}.$$

Производительность модели ОС по сравнению с однопрограммной и доля системных затрат:

$$D_{multi} = M_{multi} / M_{mono} \cdot 100\ \% = \Sigma\, T_{mono} / T_{эфф} \cdot 100\ \%,$$

$$D_{sys} = T_{сист} / T_{эфф} \cdot 100\ \%.$$

Длительности системных действий по умолчанию: загрузка — 2 такта, переключение — 1, инициализация ВВ — 1, прерывание ВВ — 1, завершение — 1 такт. Они задаются в конфигурации модели и могут изменяться при проведении экспериментов.

БСА

Блок-схемы алгоритмов, реализованных в данной работе, представлены на рисунках 1–4. При построении использованы символы по ГОСТ 19.701-90: терминатор — скруглённый прямоугольник, процесс — прямоугольник, предопределённый процесс (вызов подпрограммы) — прямоугольник с двойными боковыми линиями, решение — ромб, подготовка к циклу — шестиугольник, вывод данных — параллелограмм.

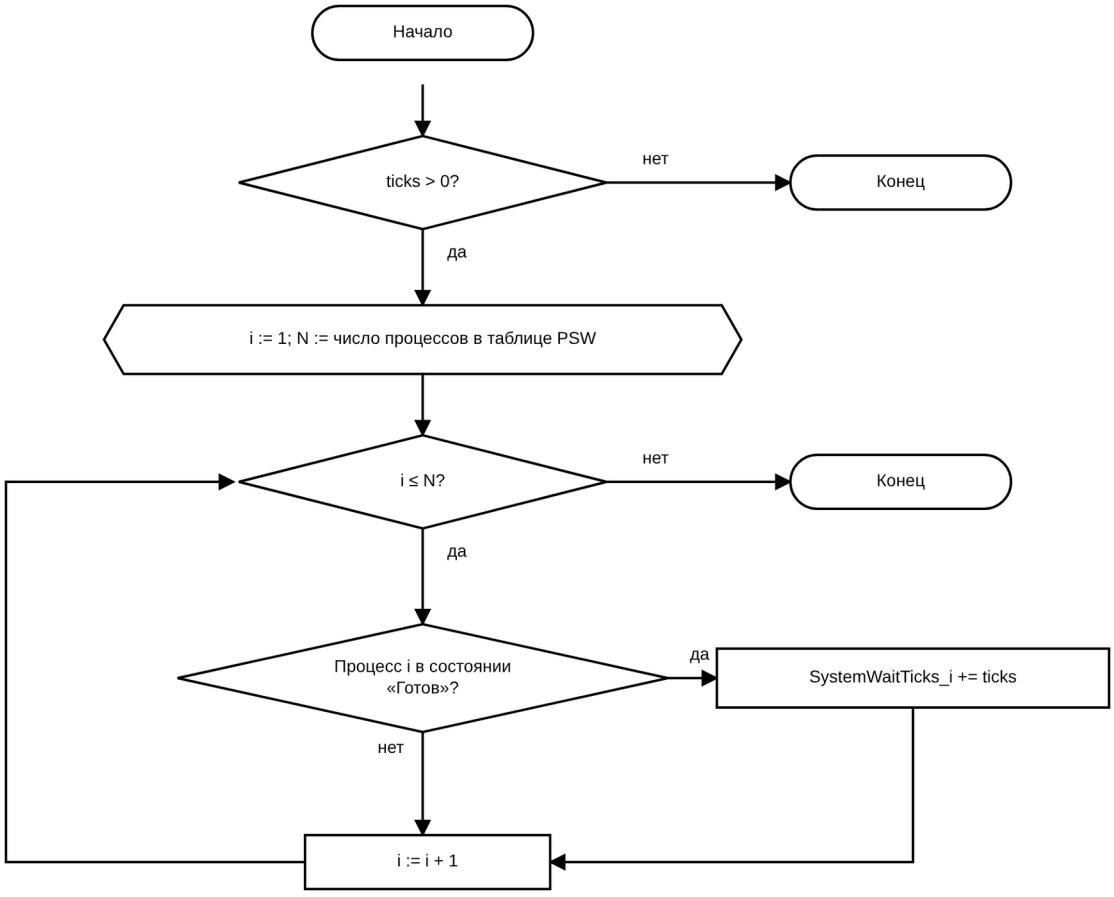


Рисунок 1 — Подпрограмма «Увеличить время готовых процессов»

Подпрограмма IncreaseReadyTime (рисунок 1) увеличивает на заданное число тактов время пребывания в системе всех процессов, находящихся в состоянии «Готов». Остальные процессы время не получают: активный

процесс в это время не ожидает, а процесс, блокированный по вводу-выводу, по правилу лабораторной работы системное время не получает. Нулевое или отрицательное приращение игнорируется.

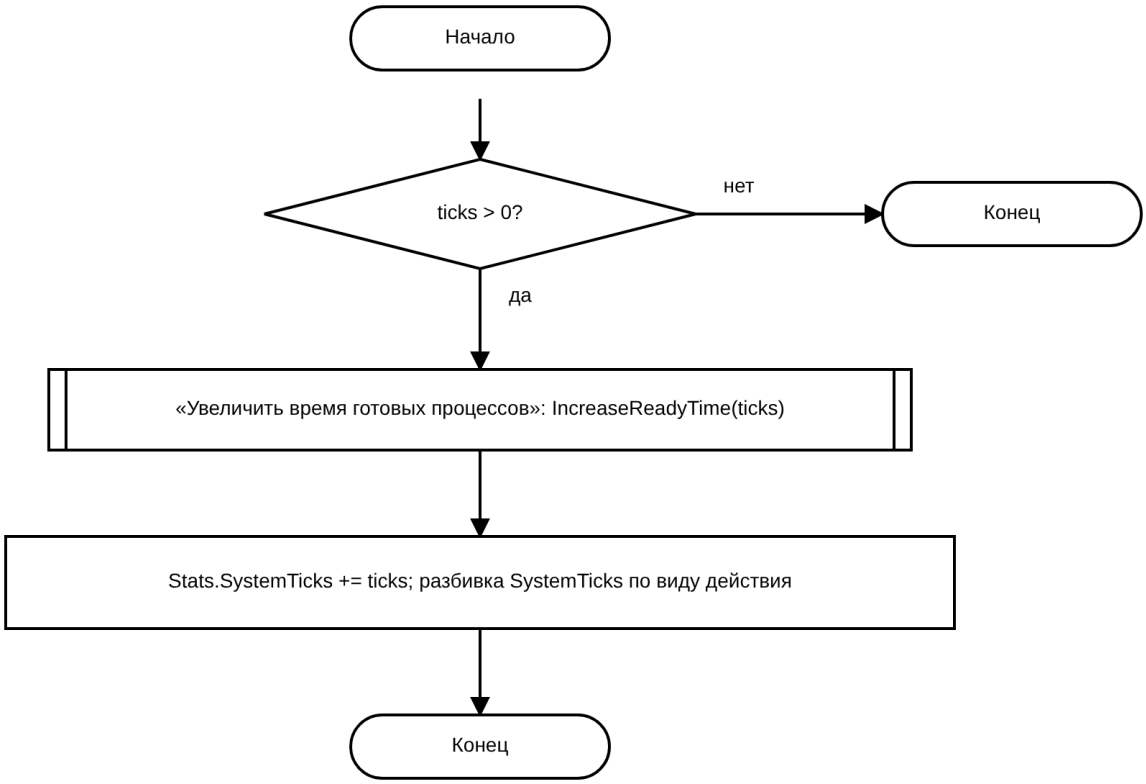


Рисунок 2 — Начисление системных затрат при выполнении действия

Начисление системных затрат (рисунок 2) выполняется регулировщиком при каждом системном действии. Сначала вызывается подпрограмма «Увеличить время готовых процессов», затем время действия один раз учитывается в суммарных системных затратах модели. Собственные затраты процесса (загрузка, переключение, инициализация ВВ, прерывание, завершение) начисляются в соответствующие поля его слова состояния.

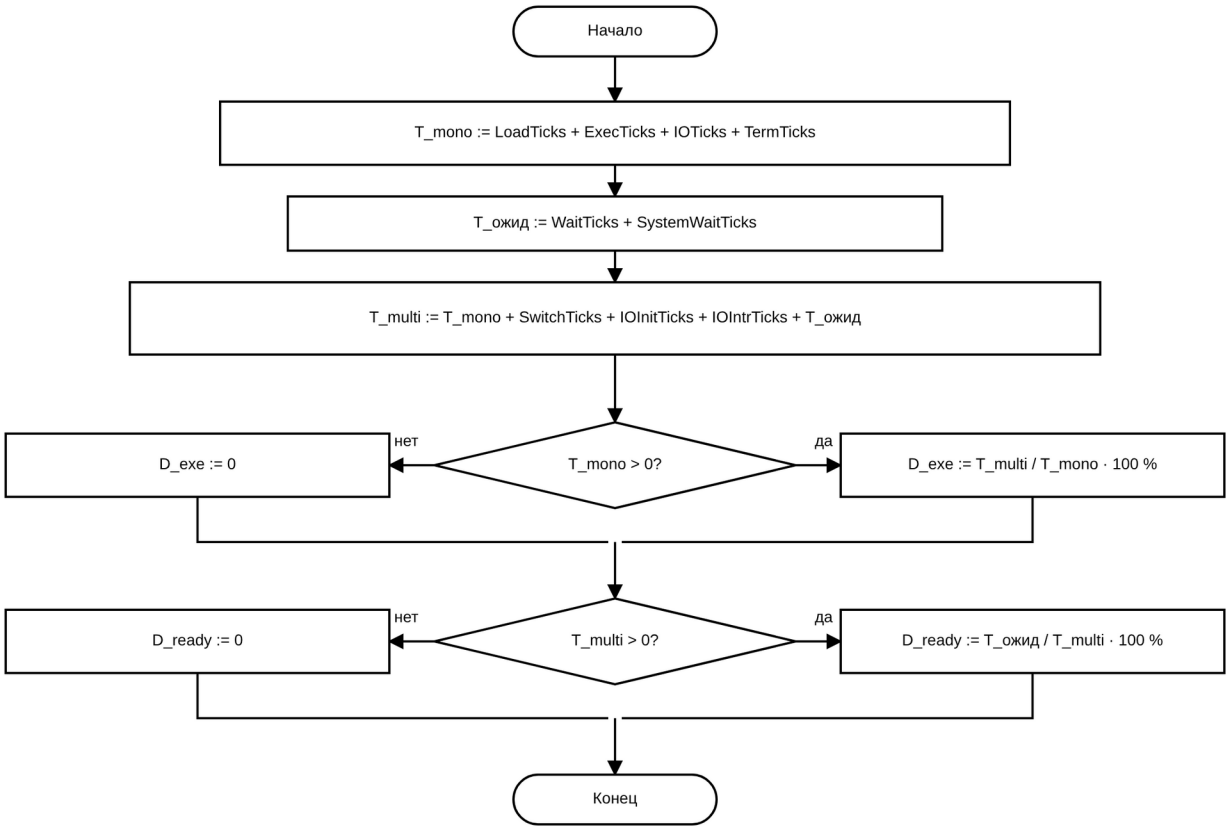


Рисунок 3 — Расчёт T_mono, T_multi, D_exe и D_ready для процесса

Расчёт времён процесса (рисунок 3) выполняется по накопленным счётчикам. Если T_mono или T_multi равны нулю, соответствующий показатель принимается равным нулю — деление на ноль не выполняется.

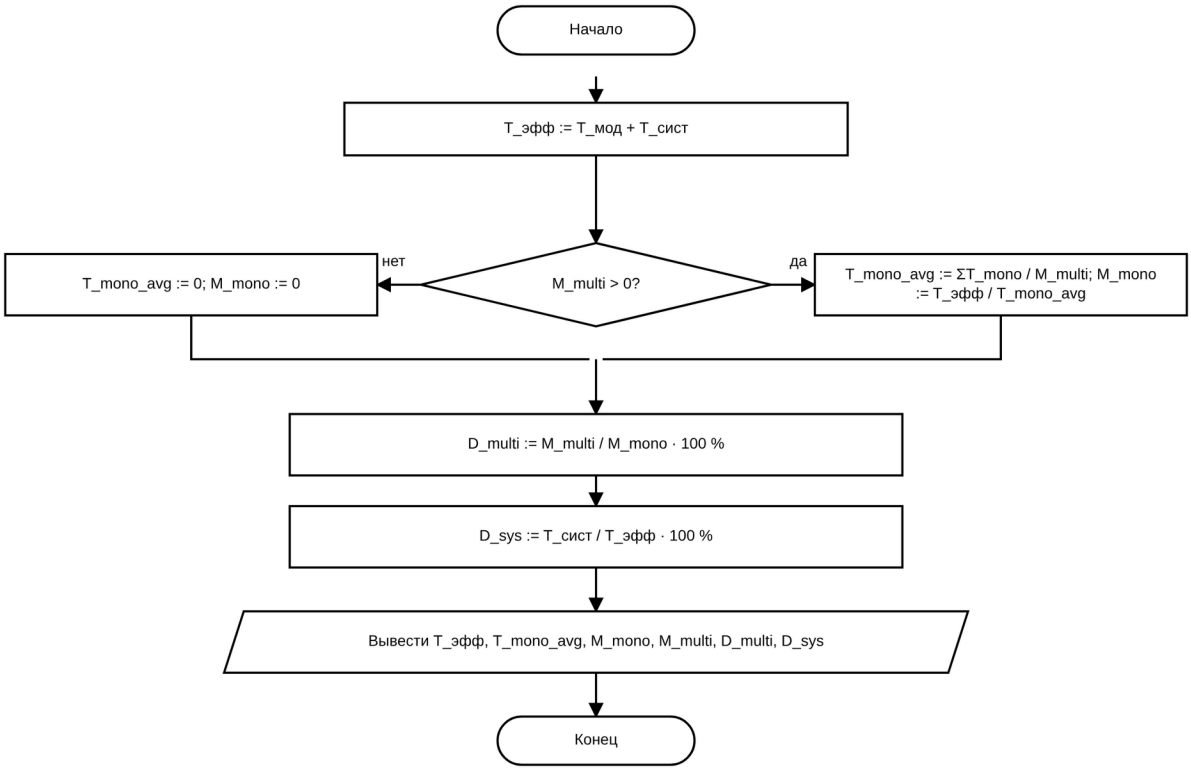


Рисунок 4 — Расчёт показателей системы: M_mono, D_multi, D_sys

Расчёт показателей системы (рисунок 4) использует накопленные суммы T_mono по завершённым процессам. При отсутствии завершённых процессов средние и возможное число заданий принимаются равными нулю.

Анализ программного кода

В лабораторной работе 7 модель дополнена расчётом времени выполнения задания в однопрограммном и мультипрограммном режимах. Добавлены длительности системных действий, счётчики системных затрат в слове состояния процесса, подпрограмма «Увеличить время готовых процессов», накопители времён и производные показатели; в интерфейс добавлены разделы сравнения режимов и затрат ОС.

1. Длительности системных действий (internal/regulator/costs.go)

Введена структура Costs с длительностями системных действий и значениями по умолчанию. Время системных действий не сдвигает модельные такты пользовательских команд, а начисляется процессам при событиях.

```
package regulator

//
------------------------------------------------------------------------------
--
// Costs — длительности системных действий модели в тактах (лабораторная работа 7).
//
// Системные действия выполняются не пользовательским процессом, а ОС.
// Их время не сдвигает модельные такты пользовательских команд, а
// начисляется процессам: собственные затраты — в поля PSW, готовым
// процессам — в SystemWaitTicks подпрограммой IncreaseReadyTime.
type Costs struct {
    Load        int // T_загр: загрузка задания
    Dispatch    int // T_перекл: выбор и активация процесса на ЦПр
    IOInit      int // T_иниц.ВВ: логическая инициализация ввода-вывода
    IOInterrupt int // T_прерыв: обслуживание прерывания по окончании ВВ
    Terminate   int // T_заверш: завершение и выгрузка задания
}

//
------------------------------------------------------------------------------
--
// DefaultCosts возвращает длительности системных действий по умолчанию.
// Значения подобраны так, чтобы системные затраты были заметны на фоне
// 10–20 команд задания, но не замедляли демонстрационную модель чрезмерно.
func DefaultCosts() Costs {
    return Costs{
        Load:        2,
        Dispatch:    1,
        IOInit:      1,
        IOInterrupt: 1,
```

```
        Terminate:   1,
    }
}
```

2. Слово состояния процесса (internal/process/psw.go)

В PSW добавлены составляющие системных затрат процесса: время загрузки LoadTicks, завершения TermTicks, переключений SwitchTicks, логической инициализации ВВ IOInitTicks, обслуживания прерываний IOIntrTicks и дополнительное ожидание SystemWaitTicks. Дополнительно ведутся счётчики системных действий процесса — DispatchCount, IORequestCount, IOCompleteCount, которые используются для проверки формул и индикации.

```
    // Системные затраты процесса (лабораторная работа 7). Это время,
    // которое процесс провёл бы в системе при обслуживании его системных
    // запросов: загрузка, выбор на ЦПр, инициализация ВВ, прерывание ВВ,
    // завершение. В отличие от WaitTicks/ExecTicks/IOTicks, эти поля
    // начисляются не каждый такт, а в моменты системных действий.
    LoadTicks       int // время загрузки задания, тактов
    TermTicks       int // время завершения задания, тактов
    SwitchTicks     int // время переключений «Готов» → «Активен», тактов
    IOInitTicks     int // время логической инициализации ВВ, тактов
    IOIntrTicks     int // время обслуживания прерываний ВВ, тактов
    SystemWaitTicks int // дополнительное ожидание из-за системных действий, тактов


    // Счётчики системных действий процесса — используются для проверки
    // формул T_mono/T_multi и для индикации.
    DispatchCount   int // сколько раз процесс выбирался на ЦПр
    IORequestCount  int // сколько операций ВВ было инициировано
    IOCompleteCount int // сколько прерываний ВВ было обслужено
}
```

3. Регулировщик (internal/regulator/regulator.go)

Подпрограмма «Увеличить время готовых процессов» реализована в регулировщике и вызывается из подпрограммы charge при каждом системном действии. charge также начисляет суммарные системные затраты модели — один раз на действие; собственные затраты процесса начисляются в его поля.

```
// IncreaseReadyTime — подпрограмма «Увеличить время готовых процессов»
// (лабораторная работа 7). Увеличивает на ticks время пребывания в системе
// всех процессов, находящихся в состоянии «Готов». Именно они ожидают,
// пока ОС выполняет системное действие.
//
// Состояния «Активен», «Инициализация IO» и «Блокирован» не изменяются:
// активный процесс в это время не ожидает, а процесс, блокированный
// по вводу-выводу, по правилу лабораторной работы системное время
// не получает.
func (r *Regulator) IncreaseReadyTime(ticks int) {
    if ticks <= 0 {
        return
    }
    for _, p := range r.Table {
        if p.State == process.StateReady {
```

```
            p.SystemWaitTicks += ticks
        }
    }
}
// charge начисляет время системного действия: увеличивает ожидание готовых
// процессов и суммарные системные затраты. Собственные затраты процесса
// (LoadTicks, SwitchTicks и т. д.) начисляет вызвавшая подпрограмма.
func (r *Regulator) charge(action stats.SystemAction, ticks int) {
    r.IncreaseReadyTime(ticks)
    if r.Stats != nil {
        r.Stats.OnSystemAction(action, ticks)
    }
}
```

Вызовы подпрограммы добавлены в обработку всех событий регулировщика:

Событие	**Собственные затраты процесса**	**Суммарные затраты**
Загрузка задания (EventLoad)	LoadTicks += Costs.Load	SystemLoad
Выбор на ЦПр (EventActivate)	SwitchTicks += Costs.Dispatch, DispatchCount++	SystemDispatch
Запрос ВВ (EventIORequest)	IOInitTicks += Costs.IOInit, IORequestCount++	SystemIOInit
Окончание ВВ (EventIOComplete)	IOIntrTicks += Costs.IOInterrupt, IOCompleteCount++	SystemIOComplete
Завершение (EventTerminate)	TermTicks += Costs.Terminate	SystemTerminate

Фрагмент обработки загрузки и завершения процесса:

```
// Системные затраты на загрузку начисляются уже готовым процессам;
// новый процесс в таблицу ещё не добавлен и это время не получает.
r.charge(stats.SystemLoad, r.Costs.Load)
p.LoadTicks += r.Costs.Load
r.charge(stats.SystemTerminate, r.Costs.Terminate)
p.TermTicks += r.Costs.Terminate
```

При начислении затрат время получают только готовые процессы. Если системное действие выполняется, пока процесс активен, инициализирует ВВ или блокирован, его T_multi не увеличивается на это действие. Так реализовано правило: в однопроцессорной модели время системных процессов добавляется ко времени всех пользовательских процессов, кроме блокированных по вводу-выводу.

4. Расчёт времён процесса (internal/stats/times.go)

В модуле статистики добавлены подпрограммы расчёта T_mono, полного времени ожидания, T_multi и производных показателей процесса.

```
// MonoTicks — время выполнения процесса в однопрограммной ОС, тактов.
func MonoTicks(p *process.PSW) int {
    if p == nil {
        return 0
    }
    return p.LoadTicks + p.ExecTicks + p.IOTicks + p.TermTicks
}
```

```
// ReadyTicks — полное время ожидания процесса в очереди готовности,
// включая дополнительные такты системных действий.
func ReadyTicks(p *process.PSW) int {
    if p == nil {
        return 0
    }
    return p.WaitTicks + p.SystemWaitTicks
}
// MultiTicks — время нахождения процесса в мультипрограммной системе, тактов.
func MultiTicks(p *process.PSW) int {
    if p == nil {
        return 0
    }
    return MonoTicks(p) +
        p.SwitchTicks +
        p.IOInitTicks +
        p.IOIntrTicks +
        ReadyTicks(p)
}
// ExecutionIncrease — увеличение времени выполнения задания по сравнению
// с однопрограммной ОС, %.
func ExecutionIncrease(p *process.PSW) float64 {
    mono := MonoTicks(p)
    if mono <= 0 {
        return 0
    }
    return float64(MultiTicks(p)) / float64(mono) * 100
}
// ReadyShare — доля времени ожидания в T_multi, %.
func ReadyShare(p *process.PSW) float64 {
    multi := MultiTicks(p)
    if multi <= 0 {
        return 0
    }
    return float64(ReadyTicks(p)) / float64(multi) * 100
}
```

5. Показатели системы (internal/stats/stats.go)

Для завершённых процессов накапливаются суммы ΣT_mono, ΣT_multi, ΣD_exe, ΣD_ready. Время системного действия учитывается один раз в SystemTicks с разбивкой по видам. На их основе вычисляются эффективное время, среднее время выполнения в однопрограммной ОС, возможное число заданий, производительность и доля системных затрат.

```
// OnSystemAction учитывает время системного действия. Вызывается
// регулировщиком один раз на каждое действие; время готовых процессов
// увеличивает отдельная подпрограмма IncreaseReadyTime.
func (s *Stats) OnSystemAction(action SystemAction, ticks int) {
    if ticks <= 0 {
        return
    }

    s.SystemTicks += ticks
    switch action {
    case SystemLoad:
        s.SystemLoadTicks += ticks
    case SystemDispatch:
        s.SystemDispatchTicks += ticks
    case SystemIOInit:
        s.SystemIOInitTicks += ticks
    case SystemIOComplete:
        s.SystemIOCompleteTicks += ticks
    case SystemTerminate:
        s.SystemTerminateTicks += ticks
```

```
    }
}
// EffectiveTime — время работы модели с учётом системных затрат:
// модельные такты плюс виртуальные такты системных действий.
func (s *Stats) EffectiveTime() int { return s.Now + s.SystemTicks }


// MonoAvg — среднее время выполнения задания в однопрограммной ОС, тактов.
func (s *Stats) MonoAvg() float64 { return average(s.SumMono, s.CompletedCount) }


// MultiAvg — среднее время нахождения завершённого процесса в системе, тактов.
func (s *Stats) MultiAvg() float64 { return average(s.SumMulti, s.CompletedCount) }


// MonoPossible — сколько заданий могло бы выполниться за EffectiveTime
// в однопрограммной ОС.
func (s *Stats) MonoPossible() float64 {
    avg := s.MonoAvg()
    if avg <= 0 {
        return 0
    }
    return float64(s.EffectiveTime()) / avg
}
// MonoPossible — сколько заданий могло бы выполниться за EffectiveTime
// в однопрограммной ОС.
func (s *Stats) MonoPossible() float64 {
    avg := s.MonoAvg()
    if avg <= 0 {
        return 0
    }
    return float64(s.EffectiveTime()) / avg
}
// Performance — производительность модели ОС по сравнению с однопрограммной, %.
// M_multi / M_mono * 100 = ΣT_mono / T_эфф * 100.
func (s *Stats) Performance() float64 {
    possible := s.MonoPossible()
    if possible <= 0 {
        return 0
    }
    return float64(s.CompletedCount) / possible * 100
}
```

6. Ядро модели (internal/os/system.go, internal/os/config.go)

Длительности системных действий задаются в Config.SystemCosts; при отсутствии значения используются значения по умолчанию. Для детерминированных тестов добавлена фиксированная длительность операции ВВ FixedIODuration и отключаемая автозагрузка AutoLoadEnabled. Загрузка задания с явно заданными параметрами вынесена в подпрограмму LoadTask, которую используют и генератор заданий, и тесты.

```
// Config — параметры конфигурации модели.
type Config struct {
    MemorySize         int // общий объём моделируемой памяти
    TickDurationMs     int // длительность одного такта моделирования (скорость
работы)
    IOMaxDuration      int // максимальная длительность операции ввода-вывода, тактов
    FixedIODuration    int // фиксированная длительность ВВ для тестов (0 — случайная)
    PrioritySizeFactor int // прирост DynamicPriority за единицу размера задания
(вариант 11)


    // SystemCosts — длительности системных действий (лабораторная работа 7).
    // nil означает значения по умолчанию regulator.DefaultCosts().
    SystemCosts *regulator.Costs
```

```
}
// LoadTask загружает задание с явно заданными параметрами. Используется
// генератором заданий и детерминированными тестами лабораторной работы 7.
// Возвращает true, если задание загружено.
func (sys *System) LoadTask(task process.Task) bool {
    sys.LastTask = LastTaskInfo{Task: task}


    // Проверка места в таблице слов состояний процессов.
    if !sys.Regulator.FreeSlot() {
        sys.LastTask.Reason = "нет свободной записи в таблице PSW"
        return false
    }


    // Новое задание загружается только при наличии свободной памяти.
    if !sys.Memory.CheckAvailable(task.Size) {
        sys.LastTask.Reason = "недостаточно свободной памяти"
        return false
    }


    // Присваиваем идентификатор и выделяем память для задания.
    sys.nextTaskID++
    task.ID = sys.nextTaskID
    if err := sys.Memory.Allocate(task); err != nil {
        sys.LastTask.Reason = err.Error()
        return false
    }
    sys.LastTask.Task = task
    sys.LastTask.Loaded = true


    // Порождаем процесс; состояние «Готов» устанавливает регулировщик.
    psw := &process.PSW{
        ID:              task.ID,
        Task:            &task,
        PC:              0,
        DynamicPriority: task.BasePriority,
    }


    return sys.Regulator.Process(psw, regulator.EventLoad)
}
```

7. Снимок состояния и интерфейс (internal/os/snapshot.go, frontend/dist)

Типы данных для индикации отделены от структур ядра префиксом UI. В снимок добавлены системные затраты Costs и показатели сравнения режимов: T_мод, T_сист, T_эфф, ΣT_mono, средние T_mono и T_multi, M_mono, M_multi, D_multi, D_sys, средние D_exe и D_ready, а также разбивка системных затрат по видам. Для каждого процесса в снимок добавлены T_mono, T_multi, T_ожид, D_exe, D_ready и составляющие системных затрат.

```
// UICostsInfo — длительности системных действий (лабораторная работа 7).
type UICostsInfo struct {
    Load        int `json:"load"`
    Dispatch    int `json:"dispatch"`
    IOInit      int `json:"ioInit"`
    IOInterrupt int `json:"ioInterrupt"`
    Terminate   int `json:"terminate"`
}
```

В интерфейс добавлены три раздела: «Параметры процессов в однопрограммном и мультипрограммном режимах» (таблица по процессам), «Сравнение с однопрограммной ОС» и «Затраты ОС на системные действия». Соответствие разделов и рассчитываемых параметров приведено в таблице.

Раздел интерфейса	**Что показывает**
Параметры процессов в двух режимах	T_mono, T_multi, D_exe, T_ожид, D_ready, переключения, инициализация и прерывания ВВ, системное ожидание для каждого процесса
Сравнение с однопрограммной ОС	ΣT_mono и среднее, среднее T_multi, T_мод / T_сист / T_эфф, M_mono / M_multi, D_multi, D_sys, средние D_exe и D_ready
Затраты ОС на системные действия	длительности T_загр, T_перекл, T_иниц.ВВ, T_прерыв, T_заверш и суммарные системные затраты

8. Автоматическая проверка (internal/os/lab7_test.go и др.)

Добавлены детерминированные тесты с известными ожидаемыми значениями: одно CPU-bound задание, одно IO-bound задание, два задания с перекрытием вычислений и ввода-вывода, нулевые системные затраты. Отдельные тесты проверяют подпрограмму IncreaseReadyTime (время получают только готовые процессы) и формулы модуля статистики, включая граничные случаи с нулевыми знаменателями. Тест на длинном случайном прогоне дополнительно проверяет согласованность накопленных затрат и времён.

Листинг программы

Файл internal/regulator/costs.go:

Длительности системных действий.

```
package regulator

//
-----------------------------------------------------------------------------
--
// Costs — длительности системных действий модели в тактах (лабораторная работа 7).
//
// Системные действия выполняются не пользовательским процессом, а ОС.
// Их время не сдвигает модельные такты пользовательских команд, а
// начисляется процессам: собственные затраты — в поля PSW, готовым
// процессам — в SystemWaitTicks подпрограммой IncreaseReadyTime.
type Costs struct {
    Load        int // T_загр: загрузка задания
```

```
    Dispatch    int // T_перекл: выбор и активация процесса на ЦПр
    IOInit      int // T_иниц.ВВ: логическая инициализация ввода-вывода
    IOInterrupt int // T_прерыв: обслуживание прерывания по окончании ВВ
    Terminate   int // T_заверш: завершение и выгрузка задания
}

//
----------------------------------------------------------------------------------
--
// DefaultCosts возвращает длительности системных действий по умолчанию.
// Значения подобраны так, чтобы системные затраты были заметны на фоне
// 10–20 команд задания, но не замедляли демонстрационную модель чрезмерно.
func DefaultCosts() Costs {
    return Costs{
        Load:        2,
        Dispatch:    1,
        IOInit:      1,
        IOInterrupt: 1,
        Terminate:   1,
    }
}
```

Файл internal/stats/times.go:

Расчёт времён процесса в двух режимах.

```
package stats

import "os_model/internal/process"

//
----------------------------------------------------------------------------------
--
// Расчёт времён процесса для сравнения однопрограммного и
// мультипрограммного режимов (лабораторная работа 7).
//
// T_mono — время выполнения задания в однопрограммной ОС: загрузка,
//          выполнение команд, ввод-вывод и завершение.
// T_multi — время нахождения процесса в мультипрограммной системе:
//           к T_mono добавляются переключения, логическая инициализация
//           ВВ, обслуживание прерываний и ожидание в очереди готовности.

// MonoTicks — время выполнения процесса в однопрограммной ОС, тактов.
func MonoTicks(p *process.PSW) int {
    if p == nil {
        return 0
    }
    return p.LoadTicks + p.ExecTicks + p.IOTicks + p.TermTicks
}

// ReadyTicks — полное время ожидания процесса в очереди готовности,
// включая дополнительные такты системных действий.
func ReadyTicks(p *process.PSW) int {
    if p == nil {
        return 0
    }
    return p.WaitTicks + p.SystemWaitTicks
}

// MultiTicks — время нахождения процесса в мультипрограммной системе, тактов.
func MultiTicks(p *process.PSW) int {
    if p == nil {
        return 0
    }
    return MonoTicks(p) +
        p.SwitchTicks +
```

```
        p.IOInitTicks +
        p.IOIntrTicks +
        ReadyTicks(p)
}


// ExecutionIncrease — увеличение времени выполнения задания по сравнению
// с однопрограммной ОС, %.
func ExecutionIncrease(p *process.PSW) float64 {
    mono := MonoTicks(p)
    if mono <= 0 {
        return 0
    }
    return float64(MultiTicks(p)) / float64(mono) * 100
}


// ReadyShare — доля времени ожидания в T_multi, %.
func ReadyShare(p *process.PSW) float64 {
    multi := MultiTicks(p)
    if multi <= 0 {
        return 0
    }
    return float64(ReadyTicks(p)) / float64(multi) * 100
}
```

Файл internal/process/psw.go:

Слово состояния процесса с системными затратами.

```
package process


// State — состояние процесса в модели.
//
// Список возможных состояний по лабораторной работе 5:
// «Готов», «Активен», «Инициализация IO», «Блокирован».
// StateNone используется при выгрузке процесса (запись в таблице удаляется).
type State int


//
----------------------------------------------------------------------------------
--
const (
    StateNone      State = iota // процесс не порождён / выгружен
    StateReady                  // «Готов»: находится в очереди планировщика
    StateActive                 // «Активен»: выполняется на центральном процессоре
    StateInitIO                 // «Инициализация IO»: инициализируется операция
ввода-вывода
    StateBlockedIO              // «Блокирован»: ожидает окончания операции ввода-
вывода
)


//
----------------------------------------------------------------------------------
--
// PSW (Process Status Word, слово состояния процесса) описывает
// динамический процесс, порождённый заданием.
//
// Все изменения состояния процесса выполняет только регулировщик
// (подпрограмма Process), остальные модули лишь читают эти поля.
// Ход операции ввода-вывода (номер процессора ВВ и счётчик тактов)
// хранится в состоянии процессора ВВ — см. пакет io.
type PSW struct {
    ID              int   // идентификатор процесса
    Task            *Task // ссылка на задание, породившее процесс
    PC              int   // счётчик команд
    State           State // текущее состояние
    DynamicPriority int   // текущий (динамический) приоритет
```

```
    // Статистика времени процесса (лабораторная работа 6): заполняется
    // ядром модели каждый такт по текущему состоянию процесса.
    LoadedAt  int // такт модели, в котором процесс загружен
    WaitTicks int // время ожидания в очереди готовности, тактов
    ExecTicks int // время выполнения на центральном процессоре, тактов
    IOTicks   int // время в состоянии «Инициализация IO» и «Блокирован», тактов


    // Системные затраты процесса (лабораторная работа 7). Это время,
    // которое процесс провёл бы в системе при обслуживании его системных
    // запросов: загрузка, выбор на ЦПр, инициализация ВВ, прерывание ВВ,
    // завершение. В отличие от WaitTicks/ExecTicks/IOTicks, эти поля
    // начисляются не каждый такт, а в моменты системных действий.
    LoadTicks       int // время загрузки задания, тактов
    TermTicks       int // время завершения задания, тактов
    SwitchTicks     int // время переключений «Готов» → «Активен», тактов
    IOInitTicks     int // время логической инициализации ВВ, тактов
    IOIntrTicks     int // время обслуживания прерываний ВВ, тактов
    SystemWaitTicks int // дополнительное ожидание из-за системных действий, тактов


    // Счётчики системных действий процесса — используются для проверки
    // формул T_mono/T_multi и для индикации.
    DispatchCount   int // сколько раз процесс выбирался на ЦПр
    IORequestCount  int // сколько операций ВВ было инициировано
    IOCompleteCount int // сколько прерываний ВВ было обслужено
}
```

Файл internal/regulator/regulator.go:

Регулировщик и подпрограмма «Увеличить время готовых процессов».

```
// Package regulator реализует регулировщик — часть ОС, отвечающую за
// состояния процессов, и включает планировщик (Planner).
//
// Только регулировщик имеет право изменять состояния процессов: остальные
// модули модели (центральный процессор, подсистема ввода-вывода) лишь
// обращаются к подпрограмме Process, сообщая о произошедшем событии.
// Планировщик решает одну задачу — выбор следующего процесса из списка
// готовности; состояния он не изменяет.
// Регулировщик переводит процесс в новое состояние и выполняет связанные
// с этим изменения общих данных модели: очереди готовности, таблицы слов
// состояний процессов, памяти и счётчика завершённых заданий.
package regulator


import (
    "os_model/internal/memory"
    "os_model/internal/process"
    "os_model/internal/stats"
)


//
------------------------------------------------------------------------------
--
// Event — событие модели, по которому регулировщик изменяет состояние процесса.
type Event int


//
------------------------------------------------------------------------------
--
const (
    EventLoad       Event = iota // загрузка задания: создан процесс в состоянии
«Готов»
    EventActivate                // выбор планировщика: «Готов» → «Активен»
    EventIORequest               // инициализация ВВ: «Активен» → «Инициализация IO»
    EventIOStart                 // процессор ВВ начал работу: «Инициализация IO» →
«Блокирован»
```

```
    EventIOComplete              // прерывание от ВВ: «Блокирован» → «Готов»
    EventTerminate               // завершение задания: выгрузка процесса
)

//
-----------------------------------------------------------------------------
--
// Regulator — регулировщик модели. Включает планировщик (Planner),
// выбирающий следующий процесс из очереди готовности.
type Regulator struct {
    Planner *Planner // планировщик: очередь готовности и диспетчеризация

    Memory       *memory.Allocator // супервизор памяти: освобождается при завершении
    MaxProcesses int               // число записей в таблице слов состояний процессов

    // Stats — модуль статистики: накапливает входные и выходные параметры
    // процессов (лабораторная работа 6). Может быть nil.
    Stats *stats.Stats

    // Costs — длительности системных действий (лабораторная работа 7).
    Costs Costs

    Table     []*process.PSW // таблица слов состояний процессов (резидентные
процессы)
    Completed int            // число выполненных (завершённых) заданий
}

//
-----------------------------------------------------------------------------
--
// Process — основная подпрограмма регулировщика: обрабатывает событие event
// для процесса p, изменяя его состояние и общие данные модели.
// Возвращает false, если событие недопустимо для текущего состояния процесса.
func (r *Regulator) Process(p *process.PSW, event Event) bool {
    switch event {
    case EventLoad:
        // Загрузка задания: нужна свободная запись в таблице PSW.
        if !r.FreeSlot() {
            return false
        }
        // Системные затраты на загрузку начисляются уже готовым процессам;
        // новый процесс в таблицу ещё не добавлен и это время не получает.
        r.charge(stats.SystemLoad, r.Costs.Load)
        p.LoadTicks += r.Costs.Load
        p.State = process.StateReady
        r.Table = append(r.Table, p)
        r.Planner.Add(p)
        if r.Stats != nil {
            r.Stats.OnLoad(p)
        }

    case EventActivate:
        // Планировщик выбрал процесс для выполнения на процессоре.
        if p.State != process.StateReady {
            return false
        }
        p.State = process.StateActive
        r.Planner.Remove(p)
        // Время выбора на ЦПр получают остальные готовые процессы;
        // выбранный процесс уже переведён в «Активен».
        r.charge(stats.SystemDispatch, r.Costs.Dispatch)
        p.SwitchTicks += r.Costs.Dispatch
        p.DispatchCount++

    case EventIORequest:
        // Задание инициализирует операцию ввода-вывода: слово состояния
```

```
        // процесса сохраняется, процесс переводится в «Инициализация IO».
        if p.State != process.StateActive {
            return false
        }
        p.State = process.StateInitIO
        r.charge(stats.SystemIOInit, r.Costs.IOInit)
        p.IOInitTicks += r.Costs.IOInit
        p.IORequestCount++


    case EventIOStart:
        // Процессор ВВ приступил к выполнению операции.
        if p.State != process.StateInitIO {
            return false
        }
        p.State = process.StateBlockedIO


    case EventIOComplete:
        // Сигнал прерывания от процессора ВВ: процесс снова готов
        // к выполнению и помещается в очередь готовности.
        if p.State != process.StateBlockedIO {
            return false
        }
        // Прерывание обслуживается, пока процесс ещё «Блокирован»,
        // поэтому собственное время ожидания он не получает.
        r.charge(stats.SystemIOComplete, r.Costs.IOInterrupt)
        p.IOIntrTicks += r.Costs.IOInterrupt
        p.IOCompleteCount++
        p.State = process.StateReady
        r.Planner.Add(p)


    case EventTerminate:
        // Завершение задания: выгрузка процесса.
        if p.State != process.StateActive {
            return false
        }
        r.charge(stats.SystemTerminate, r.Costs.Terminate)
        p.TermTicks += r.Costs.Terminate
        r.Planner.Remove(p)
        r.Memory.Free(*p.Task)
        r.removeFromTable(p)
        p.State = process.StateNone
        r.Completed++
        if r.Stats != nil {
            r.Stats.OnComplete(p)
        }


    default:
        return false
    }


    return true
}


//
-----------------------------------------------------------------------------
--
// IncreaseReadyTime — подпрограмма «Увеличить время готовых процессов»
// (лабораторная работа 7). Увеличивает на ticks время пребывания в системе
// всех процессов, находящихся в состоянии «Готов». Именно они ожидают,
// пока ОС выполняет системное действие.
//
// Состояния «Активен», «Инициализация IO» и «Блокирован» не изменяются:
// активный процесс в это время не ожидает, а процесс, блокированный
// по вводу-выводу, по правилу лабораторной работы системное время
// не получает.
func (r *Regulator) IncreaseReadyTime(ticks int) {
    if ticks <= 0 {
```

```
        return
    }
    for _, p := range r.Table {
        if p.State == process.StateReady {
            p.SystemWaitTicks += ticks
        }
    }
}

//
-----------------------------------------------------------------------------
--
// charge начисляет время системного действия: увеличивает ожидание готовых
// процессов и суммарные системные затраты. Собственные затраты процесса
// (LoadTicks, SwitchTicks и т. д.) начисляет вызвавшая подпрограмма.
func (r *Regulator) charge(action stats.SystemAction, ticks int) {
    r.IncreaseReadyTime(ticks)
    if r.Stats != nil {
        r.Stats.OnSystemAction(action, ticks)
    }
}

//
-----------------------------------------------------------------------------
--
// Terminate — подпрограмма завершения процесса, используемая процессором
// при выполнении команды завершения задания.
func (r *Regulator) Terminate(p *process.PSW) {
    r.Process(p, EventTerminate)
}

//
-----------------------------------------------------------------------------
--
// FreeSlot сообщает, есть ли свободная запись в таблице слов состояний процессов.
func (r *Regulator) FreeSlot() bool {
    return len(r.Table) < r.MaxProcesses
}

//
-----------------------------------------------------------------------------
--
// removeFromTable удаляет запись процесса из таблицы слов состояний процессов.
func (r *Regulator) removeFromTable(p *process.PSW) {
    for i, t := range r.Table {
        if t == p {
            r.Table = append(r.Table[:i], r.Table[i+1:]...)
            return
        }
    }
}
```

Файл internal/stats/stats.go:

Накопители и показатели сравнения режимов.

```
// Package stats — модуль сбора статистики модели операционной системы.
//
// Статистика ведётся для анализа эффективности метода распределения времени
// центрального процессора (лабораторная работа 6): времена ожидания и
// выполнения процессов, длины очередей, полезная загрузка процессора,
// производительность системы и средние значения параметров процессов.
//
// В лабораторной работе 7 добавлен расчёт времени выполнения задания в
// однопрограммном и мультипрограммном режимах, системных затрат и
// производительности по сравнению с однопрограммной ОС.
//
// Времена процесса (ожидание, выполнение, ввод-вывод) хранятся в слове
```

```
// состояния процесса — см. поля PSW в пакете process. Пакет stats хранит
// суммарные значения по системе и накопители для вычисления средних.
package stats

import "os_model/internal/process"

//
----------------------------------------------------------------------------------
--
// HistoryLimit — сколько последних процессов, выполнявшихся на процессоре,
// хранится для индикации (лабораторная работа 6).
const HistoryLimit = 16

//
----------------------------------------------------------------------------------
--
// SystemAction — вид системного действия, время которого учитывается
// в лабораторной работе 7.
type SystemAction int

const (
    SystemLoad       SystemAction = iota // загрузка задания
    SystemDispatch                       // выбор процесса на ЦПр
    SystemIOInit                         // логическая инициализация ВВ
    SystemIOComplete                     // обслуживание прерывания ВВ
    SystemTerminate                      // завершение задания
)

//
----------------------------------------------------------------------------------
--
// Stats — накопленная статистика модели.
type Stats struct {
    Now int // текущий такт модели (обновляется подпрограммой Collect)

    // Центральный процессор.
    CPUBusy int // тактов процессор выполнял команды активного процесса
    CPUIdle int // тактов процессор простаивал

    // Очередь готовности.
    ReadySum int // сумма длин очереди готовности по тактам
    ReadyMax int // максимальная длина очереди за время моделирования

    // Процессоры ввода-вывода.
    IOBusySum int // сумма занятых процессоров ВВ по тактам
    IOBusyMax int // максимум одновременно занятых процессоров ВВ

    // Последовательность выполнения процессов на ЦПр (до HistoryLimit
    // последних номеров).
    ExecHistory []int

    // Входные параметры всех загруженных заданий (накопители средних).
    LoadedCount     int
    SumSize         int
    SumCommands     int
    SumIOPercent    int
    SumBasePriority int

    // Выходные параметры завершённых процессов (накопители средних).
    CompletedCount int
    SumWait        int
    SumExec        int
    SumIO          int
    SumTurnaround  int
```

```
    // Системные затраты (лабораторная работа 7). SystemTicks — суммарное
    // время системных действий, учтённое один раз на каждое действие.
    // Остальные поля — разбивка по видам действий для индикации и тестов.
    SystemTicks           int
    SystemLoadTicks       int
    SystemDispatchTicks   int
    SystemIOInitTicks     int
    SystemIOCompleteTicks int
    SystemTerminateTicks  int


    // Накопители выходных параметров завершённых процессов для сравнения
    // однопрограммного и мультипрограммного режимов (лабораторная работа 7).
    SumMono   int     // Σ T_mono завершённых процессов
    SumMulti  int     // Σ T_multi завершённых процессов
    SumDExe   float64 // Σ D_exe завершённых процессов, %
    SumDReady float64 // Σ D_ready завершённых процессов, %
}


//
------------------------------------------------------------------------------
--
// OnLoad регистрирует загруженное задание: запоминает такт загрузки
// и накапливает входные параметры для средних. Вызывается регулировщиком.
func (s *Stats) OnLoad(p *process.PSW) {
    p.LoadedAt = s.Now


    s.LoadedCount++
    s.SumSize += p.Task.Size
    s.SumCommands += p.Task.TotalCommands
    s.SumIOPercent += p.Task.IOPercent
    s.SumBasePriority += p.Task.BasePriority
}


//
------------------------------------------------------------------------------
--
// OnComplete накапливает выходные параметры завершённого процесса:
// времена ожидания, выполнения, ввода-вывода и оборотное время.
// Вызывается регулировщиком при выгрузке процесса.
func (s *Stats) OnComplete(p *process.PSW) {
    s.CompletedCount++
    s.SumWait += p.WaitTicks
    s.SumExec += p.ExecTicks
    s.SumIO += p.IOTicks
    s.SumTurnaround += s.Now - p.LoadedAt


    // Параметры сравнения режимов (лабораторная работа 7).
    s.SumMono += MonoTicks(p)
    s.SumMulti += MultiTicks(p)
    s.SumDExe += ExecutionIncrease(p)
    s.SumDReady += ReadyShare(p)
}


//
------------------------------------------------------------------------------
--
// OnDispatch запоминает номер процесса, выбранного планировщиком для
// выполнения на центральном процессоре. Хранятся последние HistoryLimit
// номеров — «последовательность выполнения процессов на процессоре».
func (s *Stats) OnDispatch(id int) {
    s.ExecHistory = append(s.ExecHistory, id)
    if len(s.ExecHistory) > HistoryLimit {
        s.ExecHistory = s.ExecHistory[len(s.ExecHistory)-HistoryLimit:]
    }
}
```

```
//
-------------------------------------------------------------------------------
--
// OnSystemAction учитывает время системного действия. Вызывается
// регулировщиком один раз на каждое действие; время готовых процессов
// увеличивает отдельная подпрограмма IncreaseReadyTime.
func (s *Stats) OnSystemAction(action SystemAction, ticks int) {
    if ticks <= 0 {
        return
    }

    s.SystemTicks += ticks
    switch action {
    case SystemLoad:
        s.SystemLoadTicks += ticks
    case SystemDispatch:
        s.SystemDispatchTicks += ticks
    case SystemIOInit:
        s.SystemIOInitTicks += ticks
    case SystemIOComplete:
        s.SystemIOCompleteTicks += ticks
    case SystemTerminate:
        s.SystemTerminateTicks += ticks
    }
}

//
-------------------------------------------------------------------------------
--
// Collect фиксирует состояние модели за один такт: занятость процессора,
// длины очередей и времена процессов по их текущим состояниям.
// Вызывается ядром каждый такт после диспетчеризации и до выполнения
// команды. Состояния процессов не изменяются.
func (s *Stats) Collect(now int, cpuBusy bool, table []*process.PSW, readyLen, ioBusy
int) {
    s.Now = now

    // Занятость центрального процессора.
    if cpuBusy {
        s.CPUBusy++
    } else {
        s.CPUIdle++
    }

    // Длина очереди готовности: сумма для средней и максимум.
    s.ReadySum += readyLen
    if readyLen > s.ReadyMax {
        s.ReadyMax = readyLen
    }

    // Занятые процессоры ввода-вывода: сумма и максимум.
    s.IOBusySum += ioBusy
    if ioBusy > s.IOBusyMax {
        s.IOBusyMax = ioBusy
    }

    // Время процессов: каждый такт процесс проводит ровно в одном
    // состоянии — ожидания, выполнения или ввода-вывода.
    for _, p := range table {
        switch p.State {
        case process.StateReady:
            p.WaitTicks++
        case process.StateActive:
            p.ExecTicks++
        case process.StateInitIO, process.StateBlockedIO:
            p.IOTicks++
```

```
        }
    }
}

//
-------------------------------------------------------------------------------
--
// History возвращает копию последовательности выполнения процессов на ЦПр.
func (s *Stats) History() []int {
    history := make([]int, len(s.ExecHistory))
    copy(history, s.ExecHistory)
    return history
}

//
-------------------------------------------------------------------------------
--
// Средние значения и производные показатели.

// AvgReadyQueue — средняя длина очереди готовности, процессов.
func (s *Stats) AvgReadyQueue() float64 { return average(s.ReadySum, s.Now) }

// AvgIOBusy — среднее число одновременно занятых процессоров ВВ.
func (s *Stats) AvgIOBusy() float64 { return average(s.IOBusySum, s.Now) }

// CPUUtilization — полезная загрузка процессора, %.
func (s *Stats) CPUUtilization() float64 { return percent(s.CPUBusy, s.Now) }

// Throughput — производительность системы: выполненных заданий на 1000 тактов.
func (s *Stats) Throughput() float64 { return average(s.CompletedCount*1000, s.Now) }

// AvgSize — средний размер загруженных заданий, единиц памяти.
func (s *Stats) AvgSize() float64 { return average(s.SumSize, s.LoadedCount) }

// AvgCommands — среднее число команд загруженных заданий.
func (s *Stats) AvgCommands() float64 { return average(s.SumCommands, s.LoadedCount) }

// AvgIOPercent — средний процент команд ввода-вывода загруженных заданий.
func (s *Stats) AvgIOPercent() float64 { return average(s.SumIOPercent, s.LoadedCount)
}

// AvgPriority — средний базовый приоритет загруженных заданий.
func (s *Stats) AvgPriority() float64 { return average(s.SumBasePriority,
s.LoadedCount) }

// AvgWait — среднее время ожидания завершённых процессов, тактов.
func (s *Stats) AvgWait() float64 { return average(s.SumWait, s.CompletedCount) }

// AvgExec — среднее время выполнения завершённых процессов на ЦПр, тактов.
func (s *Stats) AvgExec() float64 { return average(s.SumExec, s.CompletedCount) }

// AvgIO — среднее время ввода-вывода завершённых процессов, тактов.
func (s *Stats) AvgIO() float64 { return average(s.SumIO, s.CompletedCount) }

// AvgTurnaround — среднее оборотное время завершённых процессов, тактов.
func (s *Stats) AvgTurnaround() float64 { return average(s.SumTurnaround,
s.CompletedCount) }

//
-------------------------------------------------------------------------------
--
// Показатели сравнения однопрограммного и мультипрограммного режимов
```

```
// (лабораторная работа 7).


// EffectiveTime — время работы модели с учётом системных затрат:
// модельные такты плюс виртуальные такты системных действий.
func (s *Stats) EffectiveTime() int { return s.Now + s.SystemTicks }


// MonoAvg — среднее время выполнения задания в однопрограммной ОС, тактов.
func (s *Stats) MonoAvg() float64 { return average(s.SumMono, s.CompletedCount) }


// MultiAvg — среднее время нахождения завершённого процесса в системе, тактов.
func (s *Stats) MultiAvg() float64 { return average(s.SumMulti, s.CompletedCount) }


// MonoPossible — сколько заданий могло бы выполниться за EffectiveTime
// в однопрограммной ОС.
func (s *Stats) MonoPossible() float64 {
    avg := s.MonoAvg()
    if avg <= 0 {
        return 0
    }
    return float64(s.EffectiveTime()) / avg
}


// Performance — производительность модели ОС по сравнению с однопрограммной, %.
// M_multi / M_mono * 100 = ΣT_mono / T_эфф * 100.
func (s *Stats) Performance() float64 {
    possible := s.MonoPossible()
    if possible <= 0 {
        return 0
    }
    return float64(s.CompletedCount) / possible * 100
}


// SystemShare — доля системных затрат в эффективном времени работы модели, %.
func (s *Stats) SystemShare() float64 {
    return percent(s.SystemTicks, s.EffectiveTime())
}


// AvgDExe — среднее увеличение времени выполнения по сравнению
// с однопрограммной ОС, %.
func (s *Stats) AvgDExe() float64 { return averageFloat(s.SumDExe, s.CompletedCount) }


// AvgDReady — средняя доля ожидания в времени нахождения процесса
// в системе, %.
func (s *Stats) AvgDReady() float64 { return averageFloat(s.SumDReady,
s.CompletedCount) }


//
---------------------------------------------------------------------------------
--
// average возвращает sum/count (0 при отсутствии данных).
func average(sum, count int) float64 {
    if count == 0 {
        return 0
    }
    return float64(sum) / float64(count)
}


// averageFloat возвращает sum/count для вещественной суммы (0 при count = 0).
func averageFloat(sum float64, count int) float64 {
    if count == 0 {
        return 0
    }
    return sum / float64(count)
}
```

```
//
-------------------------------------------------------------------------------
--
// percent возвращает part/total в процентах (0 при отсутствии данных).
func percent(part, total int) float64 {
    if total == 0 {
        return 0
    }
    return float64(part) * 100 / float64(total)
}
```

Файл internal/os/config.go:

Параметры конфигурации, включая системные затраты.

```
// Package os — ядро и координатор модели операционной системы.
//
// Связывает менеджер памяти, планировщик, процессор и подсистему
// ввода-вывода, содержит главный цикл и обработчик директив оператора.
package os


import "os_model/internal/regulator"


//
-------------------------------------------------------------------------------
--
// Config — параметры конфигурации модели.
type Config struct {
    MemorySize         int // общий объём моделируемой памяти
    TickDurationMs     int // длительность одного такта моделирования (скорость
работы)
    IOMaxDuration      int // максимальная длительность операции ввода-вывода, тактов
    FixedIODuration    int // фиксированная длительность ВВ для тестов (0 — случайная)
    PrioritySizeFactor int // прирост DynamicPriority за единицу размера задания
(вариант 11)


    // SystemCosts — длительности системных действий (лабораторная работа 7).
    // nil означает значения по умолчанию regulator.DefaultCosts().
    SystemCosts *regulator.Costs
}


//
-------------------------------------------------------------------------------
--
```

Файл internal/os/system.go:

Ядро модели: загрузка задания, такт, автозагрузка.

```
package os


import (
    "math/rand/v2"
    "time"


    "os_model/internal/cpu"
    "os_model/internal/interrupt"
    "os_model/internal/io"
    "os_model/internal/memory"
    "os_model/internal/process"
    "os_model/internal/regulator"
    "os_model/internal/stats"
)
```

```
//
-------------------------------------------------------------------------------
--
// Параметры модели по умолчанию (если соответствующие поля Config не заполнены).
const (
    DefaultMemorySize     = 8192 // общий объём моделируемой памяти
    DefaultTickDurationMs = 1    // начальная длительность такта, мс
    DefaultIOMaxDuration  = 10   // максимальная длительность ввода-вывода, тактов
)


//
-------------------------------------------------------------------------------
--
// Параметры генерации заданий (значения по умолчанию на этапе компиляции;
// на этапе 8 будут читаться из файла параметров).
const (
    TaskSizeMin = 100 // размер задания, единиц памяти
    TaskSizeMax = 500


    TaskCommandsMin = 10 // число команд задания
    TaskCommandsMax = 20


    TaskIOPercentMin = 0 // процент команд ввода-вывода
    TaskIOPercentMax = 50


    TaskBasePriorityMin = 1 // базовый приоритет
    TaskBasePriorityMax = 10
)


//
-------------------------------------------------------------------------------
--
// MaxProcesses — число записей в таблице слов состояний процессов.
const MaxProcesses = 16


//
-------------------------------------------------------------------------------
--
// Границы скорости моделирования: от 0,1 до 1000 тактов в секунду.
const (
    minSpeed         = 0.1
    maxSpeed         = 1000.0
    speedStepPercent = 10
)


//
-------------------------------------------------------------------------------
--
// Directive — управляющая директива оператора.
type Directive int


const (
    DirectivePause     Directive = iota // приостановить моделирование
    DirectiveResume                     // продолжить моделирование
    DirectiveSpeedUp                    // увеличить скорость
    DirectiveSpeedDown                  // уменьшить скорость
    DirectiveQuit                       // завершить моделирование
)


//
-------------------------------------------------------------------------------
--
// LastTaskInfo — параметры последнего сгенерированного задания: выводятся
// подпрограммой индикации «Вывод параметров нового задания» (лабораторная
// работа 6). Задание может быть не загружено, если не хватило ресурсов.
type LastTaskInfo struct {
```

```
    Task   process.Task // параметры задания
    Loaded bool         // задание загружено в память
    Reason string       // причина, по которой задание не загружено
}

//
------------------------------------------------------------------------------------
--
// System — ядро модели: связывает все компоненты и содержит главный цикл.
type System struct {
    Memory     *memory.Allocator     // супервизор памяти
    Regulator  *regulator.Regulator  // регулировщик (состояния процессов,
планировщик)
    Interrupts *interrupt.Supervisor // супервизор прерываний
    CPU        *cpu.CPU              // центральный процессор
    IO         *io.Processors        // подсистема ввода-вывода (процессоры ВВ)
    Stats      *stats.Stats          // модуль статистики (лабораторная работа 6)


    Config Config // параметры конфигурации


    // LastTask — параметры последнего сгенерированного задания.
    LastTask LastTaskInfo


    Time int // глобальное время модели в тактах


    // Управление работой модели.
    Directives chan Directive // директивы оператора
    Speed      float64        // текущая скорость модели, тактов в секунду
    Paused     bool           // моделирование приостановлено
    Quit       bool           // получена директива завершения


    // AutoLoadEnabled разрешает автоматическую загрузку новых заданий,
    // когда ЦПр простаивает. В тестах отключается для детерминированных
    // прогонов фиксированного набора заданий.
    AutoLoadEnabled bool


    // OnTick вызывается каждый такт для индикации состояния модели
    // (например, для отправки снимка в интерфейс). Может быть nil.
    OnTick func(s *System)


    nextTaskID int // счётчик идентификаторов заданий
}

//
------------------------------------------------------------------------------------
--
// Init инициализирует структуры модели по значениям полей Config
// до запуска главного цикла.
func (sys *System) Init() {
    // Значения по умолчанию для незаполненной (или невалидной) конфигурации.
    if sys.Config.MemorySize <= 0 {
        sys.Config.MemorySize = DefaultMemorySize
    }
    if sys.Config.TickDurationMs <= 0 {
        sys.Config.TickDurationMs = DefaultTickDurationMs
    }
    if sys.Config.IOMaxDuration <= 0 {
        sys.Config.IOMaxDuration = DefaultIOMaxDuration
    }


    // Длительности системных действий: явно заданные или по умолчанию.
    costs := regulator.DefaultCosts()
    if sys.Config.SystemCosts != nil {
        costs = *sys.Config.SystemCosts
    }
```

```
    // Создание подсистем и их связывание.
    sys.Stats = &stats.Stats{}
    sys.Memory = &memory.Allocator{TotalSize: sys.Config.MemorySize}
    sys.Regulator = &regulator.Regulator{
        Planner:      &regulator.Planner{},
        Memory:       sys.Memory,
        MaxProcesses: MaxProcesses,
        Stats:        sys.Stats,
        Costs:        costs,
    }
    sys.Interrupts = &interrupt.Supervisor{}
    sys.IO = &io.Processors{
        Regulator:  sys.Regulator,
        Interrupts: sys.Interrupts,
    }
    sys.CPU = &cpu.CPU{
        IO:              sys.IO,
        Processes:       sys.Regulator,
        IOMaxDuration:   sys.Config.IOMaxDuration,
        FixedIODuration: sys.Config.FixedIODuration,
    }


    if sys.Directives == nil {
        sys.Directives = make(chan Directive, MaxProcesses)
    }


    sys.Time = 0
    sys.nextTaskID = 0
    sys.Paused = false
    sys.Quit = false
    sys.AutoLoadEnabled = true
    sys.setSpeed(1000.0 / float64(sys.Config.TickDurationMs))
}

//
-----------------------------------------------------------------------------------
--
// GenerateAndLoadTask — системный процесс загрузки задания: генерирует
// параметры нового задания и пытается загрузить его, если есть ресурсы
// (свободная запись в таблице слов состояний процессов и достаточная
// память). При успехе регулировщик создаёт процесс в состоянии «Готов».
// Возвращает true, если задание загружено.
func (sys *System) GenerateAndLoadTask() bool {
    // Параметры нового задания (выводятся подпрограммой индикации
    // «Вывод параметров нового задания»).
    return sys.LoadTask(sys.generateTask())
}

//
-----------------------------------------------------------------------------------
--
// LoadTask загружает задание с явно заданными параметрами. Используется
// генератором заданий и детерминированными тестами лабораторной работы 7.
// Возвращает true, если задание загружено.
func (sys *System) LoadTask(task process.Task) bool {
    sys.LastTask = LastTaskInfo{Task: task}


    // Проверка места в таблице слов состояний процессов.
    if !sys.Regulator.FreeSlot() {
        sys.LastTask.Reason = "нет свободной записи в таблице PSW"
        return false
    }


    // Новое задание загружается только при наличии свободной памяти.
    if !sys.Memory.CheckAvailable(task.Size) {
        sys.LastTask.Reason = "недостаточно свободной памяти"
```

```
        return false
    }

    // Присваиваем идентификатор и выделяем память для задания.
    sys.nextTaskID++
    task.ID = sys.nextTaskID
    if err := sys.Memory.Allocate(task); err != nil {
        sys.LastTask.Reason = err.Error()
        return false
    }
    sys.LastTask.Task = task
    sys.LastTask.Loaded = true

    // Порождаем процесс; состояние «Готов» устанавливает регулировщик.
    psw := &process.PSW{
        ID:              task.ID,
        Task:            &task,
        PC:              0,
        DynamicPriority: task.BasePriority,
    }

    return sys.Regulator.Process(psw, regulator.EventLoad)
}

//
-------------------------------------------------------------------------------------
--
// HandleDirectives обрабатывает директивы оператора:
// пауза, изменение скорости моделирования, завершение работы.
func (sys *System) HandleDirectives() {
    for {
        select {
        case d := <-sys.Directives:
            sys.applyDirective(d)
        default:
            return
        }
    }
}

//
-------------------------------------------------------------------------------------
--
// RunLoop — главный цикл модели. В каждом такте выполняется подпрограмма
// DoOneTick (алгоритм одного такта моделирования), затем обрабатываются
// директивы оператора и обновляется индикация.
func (sys *System) RunLoop() {
    // Цикл загрузки заданий для начала моделирования: загружается столько
    // заданий, сколько помещается в память и в таблицу процессов.
    if sys.AutoLoadEnabled {
        for sys.GenerateAndLoadTask() {
        }
    }

    for !sys.Quit {
        if !sys.Paused {
            sys.DoOneTick()
        } else {
            // На паузе изменения (директивы, скорость) тоже нужно показывать.
            sys.notify()
        }

        sys.HandleDirectives()

        if sys.Quit {
            sys.notify() // завершающий снимок состояния
```

```
            break
        }

        sys.sleep()
    }
}

//
------------------------------------------------------------------------------
--
// DoOneTick — выполнение одного такта моделирования в соответствии
// с алгоритмом одного такта модели ОС:
//
//    А  — инициализация цикла: выбрать процесс, сделать его активным
//         и восстановить слово состояния процесса;
//    Сб — сбор статистики за такт (времена процессов, занятость ЦПр,
//         длины очередей) — лабораторная работа 6;
//    Б1 — моделирование ЦПр: выполнить очередную команду активного процесса;
//    Б2 — моделирование процессоров ВВ: один такт операций ввода-вывода
//         и обработка прерываний от процессоров ВВ;
//    Б3 — моделирование прерываний по времени (только для алгоритма RR;
//         вариант 11 использует относительные приоритеты без вытеснения,
//         поэтому шаг отсутствует);
//    Б4 — проверка достигнутого состояния системы и вызов планировщика;
//    Б5 — обновление баз данных модели ОС;
//    Б6 — отображение изменений в состоянии системы.
func (sys *System) DoOneTick() {
    sys.Time++

    // А. Инициализация цикла.
    if sys.CPU.State == cpu.StateIdle {
        sys.dispatchProcess()
    }

    // Сб. Учёт статистики за текущий такт: состояние фиксируется после
    // диспетчеризации и до выполнения команды.
    sys.collectStats()

    // Б1. Моделирование центрального процессора.
    sys.CPU.Tick()

    // Б2. Моделирование процессоров ввода-вывода и обработка прерываний.
    sys.IO.TickIO(sys.Time)
    sys.HandleInterrupts()

    // Б3. Прерывания по времени не моделируются (см. комментарий выше).

    // Б4. Проверка достигнутого состояния системы.
    sys.CheckSystemState()

    // Б5. Обновление баз данных модели ОС.
    sys.Regulator.Planner.UpdatePriorities()

    // Б6. Отображение изменений в состоянии системы.
    sys.notify()
}

//
------------------------------------------------------------------------------
--
// dispatchProcess — выбор планировщиком следующего процесса и передача его
// на выполнение: регулировщик переводит процесс в состояние «Активен»,
// процессор восстанавливает слово состояния процесса.
func (sys *System) dispatchProcess() bool {
    p := sys.Regulator.Planner.SelectNext()
```

```
    if p == nil {
        return false
    }

    if !sys.Regulator.Process(p, regulator.EventActivate) {
        return false
    }

    sys.Stats.OnDispatch(p.ID)
    sys.CPU.ContextSwitch(p)
    return true
}

//
---------------------------------------------------------------------------------------
--
// collectStats — подпрограмма сбора статистики за такт (лабораторная
// работа 6): занятость центрального процессора, длины очередей и времена
// процессов по их состояниям.
func (sys *System) collectStats() {
    sys.Stats.Collect(
        sys.Time,
        sys.CPU.State == cpu.StateWorking,
        sys.Regulator.Table,
        len(sys.Regulator.Planner.ReadyQueue),
        len(sys.IO.Operations),
    )
}

//
---------------------------------------------------------------------------------------
--
// CheckSystemState — проверка достигнутого состояния системы (шаг Б4).
//
// Для всех процессоров: если резидентный процесс перестал быть активным
// (заблокирован или завершён), процессор освобождается; если процессор
// находится в состоянии «Ожидание», а список готовности не пуст, вызывается
// планировщик для выбора следующего процесса. Если готовых процессов нет,
// процессор переходит в состояние «Ожидание», а ОС порождает системный
// процесс загрузки нового задания, если есть ресурсы.
func (sys *System) CheckSystemState() {
    // Резидентный процесс мог перестать быть активным: его состояние
    // изменил регулировщик по событию ВВ или завершения.
    if p := sys.CPU.ActiveProcess; p != nil && p.State != process.StateActive {
        sys.CPU.Release()
    }

    if sys.CPU.State != cpu.StateIdle {
        return
    }

    // Список готовности не пуст — планировщик выбирает следующий процесс.
    if len(sys.Regulator.Planner.ReadyQueue) > 0 {
        sys.dispatchProcess()
        return
    }

    // Готовых процессов нет: ЦПр «Ожидает», ОС может загрузить одно новое
    // задание (место в таблице PSW и достаточная память проверяются
    // в GenerateAndLoadTask). В тестах автозагрузка может быть отключена.
    if sys.AutoLoadEnabled && sys.GenerateAndLoadTask() {
        sys.dispatchProcess()
    }
}
```

```
//
-------------------------------------------------------------------------------
--
// HandleInterrupts — супервизор прерываний: обрабатывает накопленные сигналы.
// Обработка сводится к вызову регулировщика, который изменяет состояния
// процессов (прерывание от процессора ВВ переводит процесс в «Готов»).
func (sys *System) HandleInterrupts() {
    for _, signal := range sys.Interrupts.ExtractAll() {
        switch signal.Type {
        case interrupt.IOComplete:
            sys.Regulator.Process(signal.Process, regulator.EventIOComplete)
        }
    }
}

//
-------------------------------------------------------------------------------
--
// generateTask генерирует параметры нового задания в заданных диапазонах
// (без идентификатора: идентификатор присваивается загруженному заданию).
func (sys *System) generateTask() process.Task {
    return process.Task{
        Size:          randomInRange(TaskSizeMin, TaskSizeMax),
        TotalCommands: randomInRange(TaskCommandsMin, TaskCommandsMax),
        IOPercent:     randomInRange(TaskIOPercentMin, TaskIOPercentMax),
        BasePriority:  randomInRange(TaskBasePriorityMin, TaskBasePriorityMax),
    }
}

//
-------------------------------------------------------------------------------
--
// applyDirective выполняет одну директиву оператора.
func (sys *System) applyDirective(d Directive) {
    switch d {
    case DirectivePause:
        sys.Paused = true
    case DirectiveResume:
        sys.Paused = false
    case DirectiveSpeedUp:
        sys.changeSpeed(speedStepPercent)
    case DirectiveSpeedDown:
        sys.changeSpeed(-speedStepPercent)
    case DirectiveQuit:
        sys.Quit = true
    }
}

//
-------------------------------------------------------------------------------
--
// changeSpeed изменяет скорость модели на заданное число процентов.
func (sys *System) changeSpeed(percent int) {
    sys.setSpeed(sys.Speed * float64(100+percent) / 100)
}

//
-------------------------------------------------------------------------------
--
// setSpeed устанавливает скорость модели (тактов в секунду)
// в допустимых пределах.
func (sys *System) setSpeed(speed float64) {
    if speed < minSpeed {
        speed = minSpeed
    }
    if speed > maxSpeed {
        speed = maxSpeed
    }
    sys.Speed = speed
```

```
}

//
------------------------------------------------------------------------------
--
// notify вызывает обработчик индикации, если он установлен.
func (sys *System) notify() {
    if sys.OnTick != nil {
        sys.OnTick(sys)
    }
}

//
------------------------------------------------------------------------------
--
// sleep выполняет программную задержку в соответствии со скоростью модели.
func (sys *System) sleep() {
    if sys.Speed <= 0 {
        return
    }
    time.Sleep(time.Duration(float64(time.Second) / sys.Speed))
}

//
------------------------------------------------------------------------------
--
// randomInRange возвращает случайное число из диапазона [min, max].
func randomInRange(min, max int) int {
    if max <= min {
        return min
    }
    return min + rand.IntN(max-min+1)
}

//
------------------------------------------------------------------------------
--
```

Файл internal/os/lab7_test.go:

Детерминированные тесты расчёта параметров.

```
package os

import (
    "math"
    "testing"


    "os_model/internal/process"
    "os_model/internal/regulator"
)

// testCosts — фиксированные системные затраты для детерминированных тестов.
func testCosts() regulator.Costs {
    return regulator.Costs{
        Load:        2,
        Dispatch:    1,
        IOInit:      1,
        IOInterrupt: 1,
        Terminate:   1,
    }
}

// newLab7System создаёт модель с отключённой автозагрузкой и
// детерминированной длительностью операций ВВ (ровно 1 такт).
```

```
func newLab7System(t *testing.T, costs regulator.Costs) *System {
    t.Helper()


    sys := &System{Config: Config{
        MemorySize:     4096,
        TickDurationMs: 1,
        IOMaxDuration:  1,
        SystemCosts:    &costs,
    }}
    sys.Init()
    sys.AutoLoadEnabled = false
    return sys
}


// runUntilCompleted выполняет такты, пока не завершится want процессов.
func runUntilCompleted(t *testing.T, sys *System, want int) {
    t.Helper()


    const limit = 10000
    for tick := 0; tick < limit && sys.Regulator.Completed < want; tick++ {
        sys.DoOneTick()
    }
    if sys.Regulator.Completed != want {
        t.Fatalf("завершено %d процессов, ожидалось %d (такт %d)",
            sys.Regulator.Completed, want, sys.Time)
    }
}


// TestLab7SingleCPUBound проверяет простейший случай: одно задание без ВВ.
//
// Трассировка при затратах 2/1/1/1/1 и 3 командах:
//
//    загрузка: T_load = 2
//    такт 1: выбор на ЦПр T_next = 1, первая команда
//    такты 2–3: ещё две команды
//    такт 4: команда завершения (такт входит в ExecTicks)
//
//    T_mono  = 2 + 4 + 0 + 1 = 7
//    T_multi = 7 + 1 = 8
//    T_эфф   = 4 модельных такта + 4 системных = 8
func TestLab7SingleCPUBound(t *testing.T) {
    sys := newLab7System(t, testCosts())
    if !sys.LoadTask(process.Task{Size: 100, TotalCommands: 3, IOPercent: 0,
BasePriority: 5}) {
        t.Fatal("задание не загружено")
    }


    runUntilCompleted(t, sys, 1)
    stats := sys.Stats


    if got, want := stats.SumMono, 7; got != want {
        t.Errorf("T_mono = %d, ожидалось %d", got, want)
    }
    if got, want := stats.SumMulti, 8; got != want {
        t.Errorf("T_multi = %d, ожидалось %d", got, want)
    }
    if got, want := stats.EffectiveTime(), 8; got != want {
        t.Errorf("T_эфф = %d, ожидалось %d", got, want)
    }
    if got, want := stats.SystemTicks, 4; got != want {
        t.Errorf("T_сист = %d, ожидалось %d", got, want)
    }
    if got, want := stats.AvgDExe(), 100.0*8/7; math.Abs(got-want) > 1e-9 {
        t.Errorf("D_exe = %.2f, ожидалось %.2f", got, want)
    }
    if got, want := stats.Performance(), 100.0*7/8; math.Abs(got-want) > 1e-9 {
        t.Errorf("D_multi = %.2f, ожидалось %.2f", got, want)
```

```
    }
}


// TestLab7SingleIOBound проверяет одно задание, состоящее только из команд ВВ.
//
// При IOMaxDuration = 1 каждая операция ВВ занимает один такт обслуживания
// плюс такт инициализации. Для двух команд ВВ:
//
//    T_mono  = 2 (загрузка) + 3 (команды, включая завершение)
//              + 2 (такты ВВ) + 1 (завершение) = 8
//    T_multi = 8 + 3 (три выбора на ЦПр) + 2 (инициализация ВВ)
//              + 2 (прерывания ВВ) = 15
//    T_эфф   = 5 модельных тактов + 10 системных = 15
func TestLab7SingleIOBound(t *testing.T) {
    sys := newLab7System(t, testCosts())
    if !sys.LoadTask(process.Task{Size: 100, TotalCommands: 2, IOPercent: 100,
BasePriority: 5}) {
        t.Fatal("задание не загружено")
    }


    runUntilCompleted(t, sys, 1)
    stats := sys.Stats


    if got, want := stats.SumMono, 8; got != want {
        t.Errorf("T_mono = %d, ожидалось %d", got, want)
    }
    if got, want := stats.SumMulti, 15; got != want {
        t.Errorf("T_multi = %d, ожидалось %d", got, want)
    }
    if got, want := stats.EffectiveTime(), 15; got != want {
        t.Errorf("T_эфф = %d, ожидалось %d", got, want)
    }
    if got, want := stats.SystemTicks, 10; got != want {
        t.Errorf("T_сист = %d, ожидалось %d", got, want)
    }
}


// TestLab7TwoTasks проверяет перекрытие вычислений и ввода-вывода:
// два задания завершаются за время, меньшее суммы их T_mono.
func TestLab7TwoTasks(t *testing.T) {
    // Без системных затрат выигрыш от перекрытия ВВ и вычислений
    // проявляется в чистом виде.
    sys := newLab7System(t, regulator.Costs{})
    // Длительный ВВ, чтобы перекрытие с вычислениями было заметным.
    sys.CPU.FixedIODuration = 10


    // Первым выбирается задание с большим базовым приоритетом.
    if !sys.LoadTask(process.Task{Size: 100, TotalCommands: 20, IOPercent: 0,
BasePriority: 1}) {
        t.Fatal("CPU-задание не загружено")
    }
    if !sys.LoadTask(process.Task{Size: 100, TotalCommands: 4, IOPercent: 100,
BasePriority: 10}) {
        t.Fatal("IO-задание не загружено")
    }


    runUntilCompleted(t, sys, 2)
    stats := sys.Stats


    t.Logf("T_mono=%d T_multi=%d T_мод=%d T_сист=%d T_эфф=%d D_multi=%.2f%%
D_exe_avg=%.2f%% D_ready_avg=%.2f%%",
        stats.SumMono, stats.SumMulti, sys.Time, stats.SystemTicks,
        stats.EffectiveTime(), stats.Performance(), stats.AvgDExe(),
stats.AvgDReady())


    if stats.SumMono <= stats.EffectiveTime() {
```

```
        t.Errorf("нет выигрыша от мультипрограммирования: ΣT_mono=%d, T_эфф=%d",
            stats.SumMono, stats.EffectiveTime())
    }
    if got := stats.Performance(); got <= 100 {
        t.Errorf("D_multi = %.2f%%, ожидалось больше 100%%", got)
    }
    if stats.SumMulti <= stats.SumMono {
        t.Errorf("ΣT_multi=%d должно быть больше ΣT_mono=%d", stats.SumMulti,
stats.SumMono)
    }
}

// TestLab7ZeroCosts проверяет, что при нулевых системных затратах
// T_mono и T_multi не содержат системных составляющих.
func TestLab7ZeroCosts(t *testing.T) {
    sys := newLab7System(t, regulator.Costs{})
    if !sys.LoadTask(process.Task{Size: 100, TotalCommands: 3, IOPercent: 0,
BasePriority: 5}) {
        t.Fatal("задание не загружено")
    }

    runUntilCompleted(t, sys, 1)
    stats := sys.Stats

    if got, want := stats.SystemTicks, 0; got != want {
        t.Errorf("T_сист = %d, ожидалось %d", got, want)
    }
    if got, want := stats.SumMono, 4; got != want {
        t.Errorf("T_mono = %d, ожидалось %d", got, want)
    }
    if got, want := stats.SumMulti, 4; got != want {
        t.Errorf("T_multi = %d, ожидалось %d", got, want)
    }
    if got, want := stats.EffectiveTime(), sys.Time; got != want {
        t.Errorf("T_эфф = %d, ожидалось %d", got, want)
    }
    if got, want := stats.Performance(), 100.0; got != want {
        t.Errorf("D_multi = %.2f, ожидалось %.2f", got, want)
    }
}
```

Результат

В соответствии с индивидуальным заданием (вариант 11 — относительные и динамические приоритеты) в модель добавлен расчёт параметров в однопрограммном и мультипрограммном режимах. Системные затраты по умолчанию приняты равными 2/1/1/1/1 такта (загрузка, переключение, инициализация ВВ, прерывание ВВ, завершение).

Проверка программы выполнена детерминированными тестами; результаты прогона приведены в таблице.

Сценарий	**T_mono**	**T_multi**	**T_сист**	**T_эфф**	**D_multi**
Одно CPU-bound задание (3 команды, ВВ 0 %)	7	8	4	8	87,5 %

Сценарий	T_mono	T_multi	T_сист	T_эфф	D_multi
Одно IO-bound задание (2 команды, ВВ 100 %)	8	15	10	15	53,3 %
Два задания с перекрытием ВВ и вычислений (затраты 0)	66	78	0	56	117,86 %
Одно задание при нулевых системных затратах	4	4	0	4	100 %

Все ожидаемые значения совпали с фактическими, тесты пройдены. В первом сценарии T_mono = 7 тактов (2 — загрузка, 4 — выполнение команд, 1 — завершение), T_multi = 8 тактов за счёт одного переключения, поэтому производительность относительно однопрограммной ОС составляет 87,5 %: при одном задании системные затраты нечем перекрыть. Во втором сценарии к T_mono = 8 тактам добавляются три переключения, две инициализации и два прерывания ВВ — T_multi = 15 тактов. В третьем сценарии за счёт перекрытия вычислений и ввода-вывода два задания завершаются за 56 модельных тактов при ΣT_mono = 66 тактов, поэтому D_multi = 117,86 %: мультипрограммирование даёт выигрыш. При нулевых системных затратах T_mono и T_multi совпадают, а D_multi = 100 %.

На рисунке 5 приведён общий вид модели в момент модельного времени 3000 такт. Загружено 192 задания, выполнено 186; в таблице PSW находятся 6 процессов, в очереди готовности — 4, занят один процессор ВВ. Полезная загрузка ЦПр — 100 %, системные затраты — 2733 такта, эффективное время — 5733 такта.

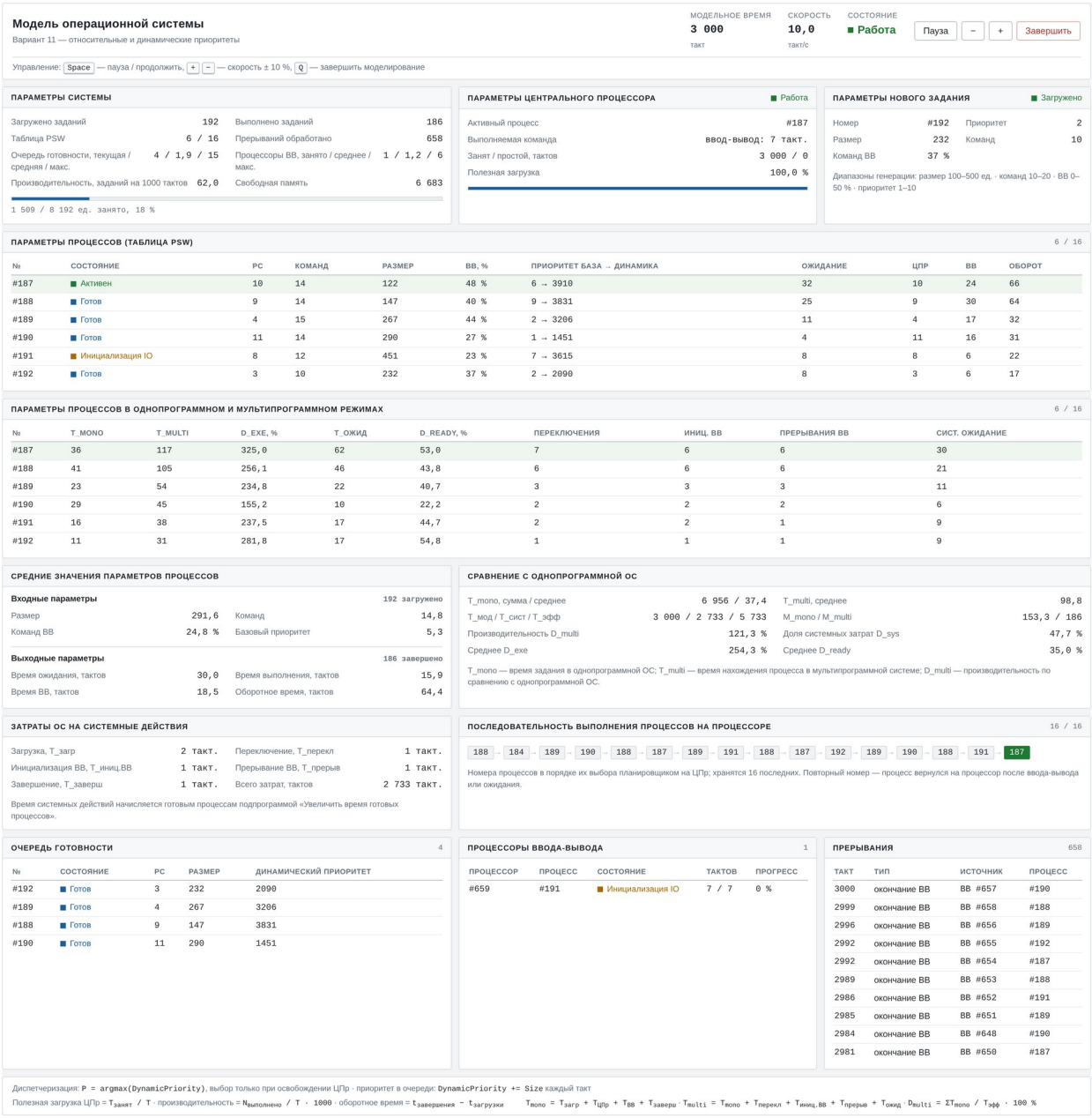
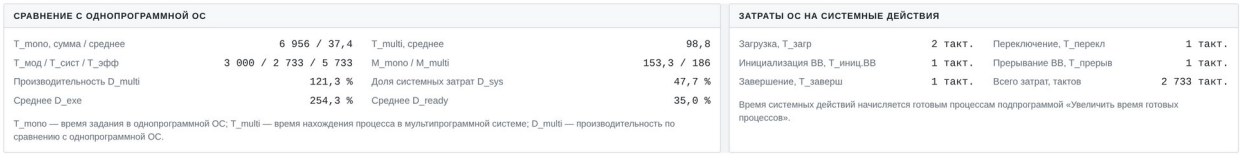


Рисунок 5 — Общий вид модели ОС с разделами сравнения режимов

На рисунке 6 показаны разделы сравнения с однопрограммной ОС и затрат на системные действия. За 5733 эффективных такта в мультипрограммном режиме выполнено 186 заданий, тогда как в однопрограммном за это же время могло выполниться M_mono = 153,3 задания; производительность D_multi = 121,3 %. Среднее время выполнения задания в однопрограммной ОС — 37,4 такта, среднее время нахождения процесса в системе — 98,8 такта, среднее увеличение времени выполнения D_exe — 254,3 %, средняя доля ожидания D_ready — 35,0 %. Доля системных затрат D_sys = 47,7 % — почти половина эффективного времени

приходится на загрузку, переключения, инициализацию и обслуживание ВВ; именно эти затраты перекрываются полезной работой других процессов в мультипрограммном режиме.



СРАВНЕНИЕ С ОДНОПРОГРАММНОЙ ОС

T_mono, сумма / среднее	6 956 / 37,4	T_multi, среднее	98,8
T_мод / T_сист / T_эфф	3 000 / 2 733 / 5 733	M_mono / M_multi	153,3 / 186
Производительность D_multi	121,3 %	Доля системных затрат D_sys	47,7 %
Среднее D_exe	254,3 %	Среднее D_ready	35,0 %

T_mono — время задания в однопрограммной ОС; T_multi — время нахождения процесса в мультипрограммной системе; D_multi — производительность по сравнению с однопрограммной ОС.

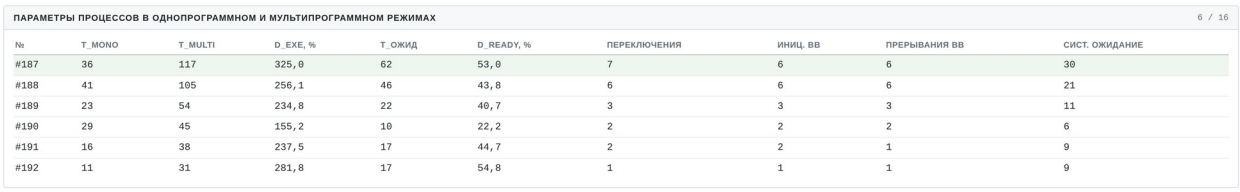
ЗАТРАТЫ ОС НА СИСТЕМНЫЕ ДЕЙСТВИЯ

Загрузка, T_загр	2 такт.	Переключение, T_перекл	1 такт.
Инициализация ВВ, T_иниц.ВВ	1 такт.	Прерывание ВВ, T_прерыв	1 такт.
Завершение, T_заверш	1 такт.	Всего затрат, тактов	2 733 такт.

Время системных действий начисляется готовым процессам подпрограммой «Увеличить время готовых процессов».

Рисунок 6 — Сравнение с однопрограммной ОС и затраты на системные действия

На рисунке 7 приведены параметры процессов в двух режимах. Для каждого процесса видны T_mono, T_multi, увеличение времени выполнения D_exe, полное время ожидания T_ожид, доля ожидания D_ready, а также составляющие системных затрат: число переключений, инициализаций и прерываний ВВ и дополнительное системное ожидание. Например, для процесса #187 T_mono = 36 тактов, T_multi = 117 тактов, D_exe = 325 %, время ожидания 62 такта (53 % от T_multi); процесс #190 выполнил 29 и 45 тактов соответственно, D_exe = 155,2 %, ожидание 10 тактов. Различие показателей объясняется разным числом операций ВВ и временем пребывания в очереди готовности: чем больше операций ВВ инициирует процесс и чем дольше он ожидает, тем сильнее T_multi превышает T_mono.



ПАРАМЕТРЫ ПРОЦЕССОВ В ОДНОПРОГРАММНОМ И МУЛЬТИПРОГРАММНОМ РЕЖИМАХ 6 / 16

№	T_MONO	T_MULTI	D_EXE, %	T_ОЖИД	D_READY, %	ПЕРЕКЛЮЧЕНИЯ	ИНИЦ. ВВ	ПРЕРЫВАНИЯ ВВ	СИСТ. ОЖИДАНИЕ
#187	36	117	325,0	62	53,0	7	6	6	30
#188	41	105	256,1	46	43,8	6	6	6	21
#189	23	54	234,8	22	40,7	3	3	3	11
#190	29	45	155,2	10	22,2	2	2	2	6
#191	16	38	237,5	17	44,7	2	2	1	9
#192	11	31	281,8	17	54,8	1	1	1	9

Рисунок 7 — Параметры процессов в однопрограммном и мультипрограммном режимах

Полученные результаты согласуются с теорией: в мультипрограммном режиме время выполнения отдельного задания увеличивается (D_exe > 100 %), поскольку процесс ожидает в очереди и обслуживается системными подпрограммами, но суммарная производительность системы возрастает за счёт перекрытия ввода-вывода и вычислений. При увеличении доли команд ВВ и числа одновременно выполняемых заданий выигрыш от мультипрограммирования растёт; при малом числе заданий системные

затраты не перекрываются и производительность может быть ниже однопрограммной (сценарий с одним CPU-bound заданием — 87,5 %).

Вывод

В ходе данной работы проведён анализ работы ПО «Модель ОС» в однопрограммном и мультипрограммном режимах. Составлена таблица составляющих времени выполнения задания: в однопрограммной ОС учитываются загрузка, выполнение команд, ввод-вывод и завершение, в мультипрограммной — дополнительно переключения, логическая инициализация ВВ, обслуживание прерываний и ожидание в очереди готовности. В модель добавлена подпрограмма «Увеличить время готовых процессов», вызываемая при системных действиях, счётчики системных затрат в слове состояния процесса и расчёт T_mono, T_multi, D_exe, D_ready, M_mono, M_multi, D_multi и D_sys. Разработанные детерминированные тесты подтвердили правильность формул: при одном CPU-bound задании производительность составила 87,5 %, при перекрытии вычислений и ввода-вывода — 117,86 %, при нулевых системных затратах — 100 %. На экранные формы выведены параметры каждого процесса в двух режимах, сравнение с однопрограммной ОС и распределение системных затрат. Получены практические навыки расчёта и анализа параметров модели операционной системы.